

ACTIVIDAD LABORATORIO NO.1
TRABAJO: ETIQUETADO MORFOSINTÁCTICO

PRESENTADO POR:
VIVIANA ANDREA BAUTISTA PULIDO
CARMEN EDILIA RICARDO GELVES
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING DIEGO OSORIO REINA

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
08 DE JUNIO
2025

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
TÍTULO DEL PROYECTO	3
DESARROLLO DEL PROYECTO	3
Primera y Segunda Parte Del Desarrollo Del Proyecto.....	3
Carga del Corpus (cargar_corpus):	3
Probabilidades de Emisión (calcular_probabilidades_emision):.....	5
Probabilidades de Transición (calcular_probabilidades_transicion):.....	6
Probabilidades Iniciales	7
Algoritmo de Viterbi (viterbi):.....	8
Etiquetado de Oraciones	9
Generación de Tablas	10
Análisis	19
Mejoras Implementadas	20
Tercera Parte Del Desarrollo Del Proyecto – Respuesta Preguntas Adicionales	22
1. ¿Es correcto el etiquetado morfosintáctico de la primera oración?	23
2. Resultado del etiquetado de la segunda oración y ¿por qué es correcto?.....	23
3. ¿Cuáles son las limitaciones del etiquetador morfosintáctico creado?	24
4. ¿Qué posibles mejoras se podrían aplicar?	24
CONCLUSIÓN DE LA ACTIVIDAD	25
BIBLIOGRAFÍA.....	26

INTRODUCCIÓN

En el campo del procesamiento del lenguaje natural (NLP), el etiquetado morfosintáctico es un componente crucial para desentrañar la estructura gramatical de las oraciones, habilitando aplicaciones como la traducción automática, el análisis de sentimientos y la extracción de información. Este proyecto tiene como objetivo desarrollar un etiquetador morfosintáctico basado en un Modelo Oculto de Markov (HMM) de bigramas, empleando el algoritmo de Viterbi para determinar la secuencia óptima de etiquetas morfosintácticas en oraciones en español, específicamente *"Habla con el enfermo grave de trasplantes."* y *"El enfermo grave habla de trasplantes."*

El enfoque se centra en utilizar exclusivamente las probabilidades de emisión y transición derivadas de un corpus etiquetado de 20 oraciones y 137 tokens (corpus_etiquetado.txt), procesado con herramientas como FreeLing y anotado según el estándar EAGLES. A diferencia de métodos más complejos que integran contexto semántico, reglas lingüísticas o modelos neuronales, este desarrollo prioriza la simplicidad y eficiencia del HMM de bigramas, optimizado con suavizado de Laplace, modelado de estados <START> y <END>, y filtrado de etiquetas relevantes. Esto permite evaluar la eficacia del algoritmo de Viterbi para asignar etiquetas precisas, como se demuestra en la validación de las oraciones objetivo (doc id=1 y doc id=2), y establece una base sólida para futuras mejoras, como la incorporación de trigramas o corpus más extensos.

TÍTULO DEL PROYECTO

Para el desarrollo de esta actividad elegimos como título del proyecto **"TRABAJO: ETIQUETADO MORFOSINTÁCTICO"**.

DESARROLLO DEL PROYECTO

Este desarrollo está orientado a resolver el problema de determinar la secuencia más probable de etiquetas morfosintácticas para oraciones en español, utilizando un Modelo Oculto de Markov (HMM) de bigramas y el algoritmo de Viterbi. El etiquetador asigna etiquetas POS (Part-of-Speech) basadas en probabilidades de emisión y transición extraídas de un corpus etiquetado (corpus_etiquetado.txt), logrando alta precisión en las oraciones objetivo: *"Habla con el enfermo grave de trasplantes."* y *"El enfermo grave habla de trasplantes."*. El enfoque HMM permite un análisis claro y eficiente del etiquetado morfosintáctico, al tiempo que establece una base para futuras mejoras, como la incorporación de contexto semántico, reglas lingüísticas, o modelos más avanzados como redes neuronales. A continuación, se detalla el desarrollo implementado, sus componentes principales, y las mejoras aplicadas.

Primera y Segunda Parte Del Desarrollo Del Proyecto

El código implementa un etiquetador morfosintáctico basado en un HMM de bigramas, utilizando el algoritmo de Viterbi para etiquetar las dos oraciones mencionadas. Además, incorpora suavizado de Laplace, modelado de estados <START> y <END>, y generación de tablas en Excel para cumplir con los requisitos del enunciado. A continuación, se describen los componentes principales del código:

Carga del Corpus (cargar_corpus):

▼ Laboratorio No. 1 Analizador Morfosintáctico con HMM

Presentado por: VIVIANA ANDREA BAUTISTA PULIDO - CARMEN EDILIA RICARDO GELVES - ALEJANDRO DE MENDOZA

Este notebook del laboratorio de "Procesamiento del Lenguaje Natural" utiliza un modelo oculto de Markov (HMM) para analizar frases y etiquetarlas morfosintácticamente utilizando un corpus POS.

```
[70] print("\n=== Punto 1: Cargar el Archivo del Corpus ===\n")
```

```
from google.colab import files
uploaded = files.upload()
corpus_file_path = list(uploaded.keys())[0]
print("Archivo cargado:", corpus_file_path)
```

Este componente del sistema tiene como funcionalidad principal la lectura del archivo `corpus_etiquetado.txt` subido a través de Google Colab. Procesa las líneas del archivo que están en formato tabulado, donde cada línea contiene un token, su lema y su etiqueta morfosintáctica (POS). Durante este proceso, se extraen únicamente el token (columna 1) y la etiqueta POS (columna 3), omitiendo tanto las líneas vacías como aquellas que comienzan con `<doc>`, que se utilizan como delimitadores de documento. A continuación, la respuesta de la consola:

```
=== Punto 1: Cargar el Archivo del Corpus ===
```

Elegir archivos `corpus_etiquetado.txt`

```
• corpus_etiquetado.txt(text/plain) - 2594 bytes, last modified: 8/6/2025 - 100% done
Saving corpus_etiquetado.txt to corpus_etiquetado (9).txt
Archivo cargado: corpus_etiquetado (9).txt
```

Ahora frente a esta acción se procede a procesar el Corpus con la siguiente imagen del código desarrollado:

```
analizador_HMM_desde_cero_funcional_mejorado.ipynb ☆ ☰
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Códigos + Código + Texto ▶ Ejecutar todas
print("\n=== Punto 2: Procesar el Corpus ===\n")

def cargar_corpus(file_path):
    """Carga el corpus desde un archivo de texto, devolviendo una lista de oraciones (cada una como lista de tuplas (token, etiqueta)) con su doc_id."""
    oraciones = []
    corpus_flat = []
    oracion_actual = []
    doc_id = None

    with open(file_path, "r", encoding="utf-8") as file:
        for linea in file:
            linea = linea.strip()
            if linea.startswith("<doc>"):
                # Finalizar la oración actual si existe
                if oracion_actual:
                    oraciones.append((doc_id, oracion_actual))
                    oracion_actual = []
                # Extraer el id del documento
                doc_id = linea.split('id=')[1].strip('<br>') if 'id=' in linea else None
                continue
            if linea == "":
                # Finalizar la oración actual si existe
                if oracion_actual:
                    oraciones.append((doc_id, oracion_actual))
                    oracion_actual = []
                continue
            datos = linea.split("\t")
            if len(datos) >= 3:
                tupla = (datos[0], datos[2])
                oracion_actual.append(tupla)
                corpus_flat.append(tupla)

        # Añadir la última oración si no está vacía
        if oracion_actual:
            oraciones.append((doc_id, oracion_actual))

    return oraciones, corpus_flat

oraciones, corpus_flat = cargar_corpus(corpus_file_path)
print("Corpus cargado. Número de oraciones:", len(oraciones))
print("Número total de tokens:", len(corpus_flat))
print("\nOración 1 (doc id=1, completa):")
for doc_id, oracion in oraciones:
    if doc_id == "1":
        print(oracion)
        break
print("\nOración 2 (doc id=2, completa):")
for doc_id, oracion in oraciones:
    if doc_id == "2":
        print(oracion)
        break
print("\nResumen de primeras 3 oraciones (primeros 5 tokens):")
for doc_id, oracion in oraciones[:3]:
    print(f"Doc id={doc_id}: {oracion[:5]} {'...' if len(oracion) > 5 else ''}")
```

El procesamiento organiza el corpus en dos estructuras paralelas. La primera es una lista de oraciones, donde cada oración se representa como una tupla compuesta por un identificador de documento (`doc_id`) y una lista de pares (`token`, `etiqueta`), respetando los límites de oración definidos en el corpus. La segunda estructura es una lista plana de tuplas (`token`, `etiqueta`) que permite calcular de forma directa las probabilidades de emisión necesarias para el modelo HMM.

Esta versión representa una mejora respecto al código original, el cual solo generaba la lista plana. Al preservar la estructura completa de las oraciones y extraer el identificador del documento, se posibilita el modelado de transiciones iniciales y finales por oración, además de permitir validaciones directas sobre oraciones específicas, como aquellas con `doc_id = 1` y `doc_id = 2`.

Como salida, el sistema imprime un resumen del procesamiento: el número total de oraciones (20), la cantidad de tokens (137), las oraciones completas correspondientes a los documentos con identificador 1 y 2, y una vista resumida de las tres primeras oraciones procesadas. A continuación, la respuesta de la consola:

```

=== Punto 2: Procesar el Corpus ===

Corpus cargado. Número de oraciones: 20
Número total de tokens: 137

Oración 1 (doc id=1, completa):

Oración 2 (doc id=2, completa):

Resumen de primeras 3 oraciones (primeros 5 tokens):
Doc id="1": [('Habla', 'VMIP350'), ('con', 'SP'), ('el', 'DA'), ('enfermo', 'NCMS000'), ('grave', 'AQ0CS00')] ...
Doc id="2": [('El', 'DA'), ('enfermo', 'NCMS000'), ('grave', 'AQ0CS00'), ('habla', 'VMIP350'), ('de', 'SP')] ...

```

Probabilidades de Emisión (calcular_probabilidades_emision):

```

print("\n=== Punto 3: Implementación de Calcular Probabilidades con Suavizado de Laplace ===\n")
import numpy as np

def calcular_probabilidades_emision(corpus):
    """
    Calcula las probabilidades de emisión P(palabra|etiqueta) con suavizado de Laplace.
    Args:
        corpus (list): Lista de tuplas (token, etiqueta) del corpus completo.
    Returns:
        dict: Diccionario con etiquetas como claves y otro diccionario (palabra: probabilidad) como valor.
    """
    emission = {}
    total_etiquetas = {}
    palabras = set(token for token, _ in corpus) # Vocabulario de palabras únicas
    k = 1 # Constante de suavizado de Laplace

    # Inicializar contadores para cada etiqueta
    for etiqueta in set(etiqueta for _, etiqueta in corpus):
        emission[etiqueta] = {}
        total_etiquetas[etiqueta] = 0

    # Contar frecuencias de palabra por etiqueta
    for token, etiqueta in corpus:
        emission[etiqueta][token] = emission[etiqueta].get(token, 0) + 1
        total_etiquetas[etiqueta] += 1

    # Aplicar suavizado de Laplace
    for etiqueta in emission:
        for palabra in palabras:
            # P(palabra|etiqueta) = (count(palabra, etiqueta) + k) / (total_count(etiqueta) + k * |vocab|)
            emission[etiqueta][palabra] = (emission[etiqueta].get(palabra, 0) + k) / (total_etiquetas[etiqueta] + k * len(palabras))

    # Calcular métricas de efectividad
    print("\n=== Métricas de Probabilidades de Emisión ===")
    suma_correcta = True
    for etiqueta in emission:
        suma = sum(emission[etiqueta].values())
        if not np.isclose(suma, 1.0, rtol=1e-5):
            print(f"Error: Suma de P(palabra|{etiqueta}) = {suma:.6f} (debería ser ~1.0)")
            suma_correcta = False
    if suma_correcta:
        print("Suma de probabilidades por etiqueta: Correcta (~1.0 para todas las etiquetas)")

    palabras_corpus = set(token for token, _ in corpus)
    palabras_cubiertas = set()
    for etiqueta in emission:
        palabras_cubiertas.update(emission[etiqueta].keys())
    cobertura = len(palabras_cubiertas) / len(palabras_corpus) * 100
    print(f"Cobertura de palabras: {cobertura:.2f}% ((len(palabras_cubiertas) de (len(palabras_corpus)) palabras)")

    entropias = []
    for etiqueta in emission:
        probs = np.array(list(emission[etiqueta].values()))
        entropia = -np.sum(probs * np.log2(probs + 1e-10))
        entropias.append(entropia)
    entropia_promedio = np.mean(entropias)
    print(f"Entropía promedio por etiqueta: {entropia_promedio:.4f} bits")

    etiquetas = set(etiqueta for _, etiqueta in corpus)
    total_combinaciones = len(palabras_corpus) * len(etiquetas)
    combinaciones_no_nulas = sum(len(emission[etiqueta]) for etiqueta in emission)
    sparsity = (1 - combinaciones_no_nulas / total_combinaciones) * 100
    print(f"Sparsity: {sparsity:.2f}% ((total_combinaciones - combinaciones_no_nulas) de (total_combinaciones) combinaciones tienen P=0)")

    return emission

```

Este módulo se encarga de calcular las probabilidades de emisión $P(\text{palabra} | \text{etiqueta})$, fundamentales para el etiquetado morfosintáctico con HMM. El cálculo se basa en contar la frecuencia con la que cada palabra aparece bajo cada etiqueta del corpus, aplicando además suavizado de Laplace. Este suavizado garantiza que todas las palabras del vocabulario tengan una probabilidad distinta de cero, incluso si no han sido observadas con una etiqueta específica durante el entrenamiento. La fórmula empleada es:

$$P(\{\text{palabra}\}|\{\text{etiqueta}\}) = (\text{count}(\text{palabra}, \text{etiqueta}) + 1) / (\text{total_count}(\text{etiqueta}) + |\text{vocab}|))$$

Esta implementación mejora el código original, el cual no contemplaba suavizado alguno. Como consecuencia, dicho código podía asignar probabilidades nulas a combinaciones de palabra y etiqueta no vistas, afectando la robustez del modelo. La versión actual resuelve este problema mediante el suavizado de Laplace, y además incorpora el cálculo de métricas adicionales para evaluar la efectividad del modelo de emisión. Entre estas métricas se encuentran la suma de probabilidades por etiqueta, la cobertura del vocabulario, la entropía promedio por etiqueta y el grado de dispersión (sparsity) de la matriz de emisión.

En su salida, el sistema genera un diccionario con las probabilidades de emisión, junto con indicadores como el 100% de cobertura (91 de 91 palabras cubiertas), una entropía promedio de 6.4595 bits por etiqueta, y un sparsity del 0.00%, resultado directo del uso del suavizado que asegura que no existan probabilidades cero en la matriz. A continuación, la respuesta de la consola:

Probabilidades de Transición (calcular_probabilidades_transicion):

```
def calcular_probabilidades_transicion(oraciones):
    """
    Calcula las probabilidades de transición P(etiqueta_i|etiqueta_{i-1}) con suavizado de Laplace, incluyendo <START> y <END>.
    Args:
        oraciones (list): Lista de tuplas (doc_id, oracion), donde oracion es una lista de tuplas (token, etiqueta).
    Returns:
        dict: Diccionario con etiquetas (incluyendo <START> y <END>) como claves y otro diccionario (etiqueta_siguiente: probabilidad) como valor.
    """
    transicion = {}
    total_transiciones = {}
    etiquetas = set(["<START>", "<END>"]) | set(etiqueta for _, oracion in oraciones for _, etiqueta in oracion) # Todas las etiquetas + <START> y <END>
    k = 1 # Constante de suavizado de Laplace

    # Inicializar contadores para todas las etiquetas
    for etiqueta in etiquetas:
        transicion[etiqueta] = {sig: 0 for sig in etiquetas} # Inicializar con 0 para todas las etiquetas siguientes
        total_transiciones[etiqueta] = 0

    # Contar transiciones dentro de cada oración
    for doc_id, oracion in oraciones:
        if not oracion: # Saltar oraciones vacías
            continue
        anterior = "<START>"
        for _, etiqueta in oracion:
            if anterior not in transicion: # Asegurar que anterior esté inicializado
                transicion[anterior] = {sig: 0 for sig in etiquetas}
                total_transiciones[anterior] = 0
            transicion[anterior][etiqueta] += 1
            total_transiciones[anterior] += 1
            anterior = etiqueta
        if anterior not in transicion: # Asegurar que la nueva etiqueta esté inicializada
            transicion[anterior] = {sig: 0 for sig in etiquetas}
            total_transiciones[anterior] = 0
        # Añadir transición al estado final <END>
        transicion[anterior]["<END>"] += 1
        total_transiciones[anterior] += 1
    # Aplicar suavizado de Laplace
    for etiqueta in transicion:
        for siguiente in transicion[etiqueta]:
            # P(etiqueta_i|etiqueta_{i-1}) = (count(etiqueta_{i-1}, etiqueta_i) + k) / (total_count(etiqueta_{i-1}) + k * len(etiquetas))
            transicion[etiqueta][siguiente] = (transicion[etiqueta][siguiente] + k) / (total_transiciones[etiqueta] + k * len(etiquetas))
    # Métricas de efectividad
    print("\n==== Métricas de Probabilidades de Transición ====")
    suma_correcta = True
    for etiqueta in transicion:
        suma = sum(transicion[etiqueta].values())
        if not np.isclose(suma, 1.0, rtol=1e-5):
            print(f"Error: Suma de P(etiqueta_i|etiqueta) = {suma:.6f} (debería ser ~1.0)")
            suma_correcta = False
    if suma_correcta:
        print("Suma de probabilidades por etiqueta anterior: Correcta (~1.0 para todas las etiquetas)")
    total_transiciones_posibles = len(etiquetas) * len(etiquetas)
    transiciones_no_nulas = sum(sum(1 for sig in transicion[etiqueta] if transicion[etiqueta][sig] > k / (total_transiciones[etiqueta] + k * len(etiquetas))) for etiqueta in transicion)
    cobertura = (transiciones_no_nulas / total_transiciones_posibles) * 100
    print(f"Cobertura de transiciones: (cobertura:.2f)% ((transiciones_no_nulas) de (total_transiciones_posibles) transiciones)")
    entropias = []
    for etiqueta in transicion:
        probs = np.array(list(transicion[etiqueta].values()))
        entropia = -np.sum(probs * np.log2(probs + 1e-10))
        entropias.append(entropia)
    entropia_promedio = np.mean(entropias)
    print(f"Entropía promedio por etiqueta anterior: (entropia_promedio:.4f) bits")
    sparsity = (1 - transiciones_no_nulas / total_transiciones_posibles) * 100
    print(f"Sparsity: (sparsity:.2f)% ((total_transiciones_posibles - transiciones_no_nulas) de (total_transiciones_posibles) transiciones tienen P=0)")
    return transicion

# Calcular probabilidades con suavizado
emision = calcular_probabilidades_emision(corpus_flat)
transicion = calcular_probabilidades_transicion(oraciones)
```

Este módulo calcula las probabilidades de transición:

$$P(etiqueta_i | etiqueta_{i-1})$$

Las cuales son fundamentales para la construcción del modelo oculto de Markov. El cálculo se realiza considerando las transiciones dentro de cada oración, respetando sus límites e incluyendo dos estados especiales: <START> al inicio y <END> al final de cada oración. Para garantizar que todas las combinaciones posibles tengan una probabilidad asignada, se aplica suavizado de Laplace, utilizando la fórmula:

$$P(etiqueta_i | etiqueta_{i-1}) = \frac{\text{count}(etiqueta_{i-1}, etiqueta_i) + 1}{\text{total_count}(etiqueta_{i-1}) + |\text{etiquetas}|}$$

Esta implementación representa una mejora significativa respecto al código original, que no modelaba los estados <START> ni <END>, ignoraba los límites de las oraciones y no aplicaba suavizado alguno. Al integrar estas características, la nueva versión permite capturar con mayor precisión el contexto sintáctico, incluso en corpus reducidos.

Además de calcular el diccionario de transiciones, el sistema genera métricas que permiten evaluar la calidad y cobertura del modelo. Por ejemplo, se observa una cobertura del 7.91% en las transiciones (76 de 961 posibles), lo cual es esperado dado el tamaño limitado del corpus. La entropía promedio por etiqueta anterior es de 4.8188 bits, y la sparsity del modelo alcanza el 92.09%, reflejando la escasez de datos en un conjunto de solo 20 oraciones.

Probabilidades Iniciales

```
print("\n5. Análisis de las Probabilidades Iniciales (tabla_iniciales.xlsx)")
try:
    # Leer tabla_iniciales.xlsx
    df_iniciales = pd.read_excel("tabla_iniciales.xlsx")

    # Ordenar por probabilidad descendente y seleccionar las 5 etiquetas más probables
    top_5_iniciales = df_iniciales.sort_values(by="Probabilidad", ascending=False).head(5)

    print("\nLas 5 etiquetas iniciales más probables:")
    for idx, row in top_5_iniciales.iterrows():
        print(f"- {row['Etiqueta']}: {row['Probabilidad']:.4f}")

    print("""
```

Las probabilidades iniciales del modelo se obtienen directamente a partir de las transiciones desde el estado especial <START>, calculadas previamente en la función que estima las probabilidades de transición. En lugar de recurrir a una función específica, el sistema reutiliza las frecuencias de las primeras etiquetas de cada oración, lo cual permite una estimación coherente con la estructura del corpus y evita duplicar cálculos innecesarios.

Esta implementación corrige un error presente en la versión original, donde se utilizaba una función llamada `calcular_probabilidades_iniciales` que asignaba de forma incorrecta una probabilidad de 1 únicamente a la primera etiqueta observada en el corpus, ignorando el resto de las oraciones.

En contraste, la nueva versión modela de forma adecuada las probabilidades iniciales considerando la frecuencia relativa de todas las etiquetas que aparecen en la primera posición de cada oración. Además, se aplica suavizado de Laplace para evitar probabilidades nulas en etiquetas no observadas en esa posición.

El resultado se exporta en una tabla (`tabla_iniciales.xlsx`) que contiene las probabilidades de inicio desde el estado <START> hacia cada etiqueta posible, brindando una representación clara y

completa de este componente del modelo. **A continuación, la respuesta de la consola para estos cálculos de etiqueta de oraciones para las dos frases:**

```

=== Punto 5: Etiquetar las Oraciones ===

Frase: ['Habla', 'con', 'el', 'enfermo', 'grave', 'de', 'trasplantes', '.']
Ruta más probable: ['VMIP3S0', 'SP', 'DA', 'NCMS000', 'AQ0CS00', 'SP', 'NCMN000', 'Fp']
Probabilidad total: 6.96740681967439e-21
Validación Oración 1: Correcta

Frase 2: ['El', 'enfermo', 'grave', 'habla', 'de', 'trasplantes', '.']
Ruta más probable 2: ['DA', 'NCMS000', 'AQ0CS00', 'VMIP3S0', 'SP', 'NCMN000', 'Fp']
Probabilidad total 2: 8.16166389485921e-18
Validación Oración 2: Correcta

```

Algoritmo de Viterbi (viterbi):

```

print("\n=== Punto 4: Implementar el Algoritmo de Viterbi ===\n")

def viterbi(frase, estados, transicion, emision):
    """
    Implementa el algoritmo de Viterbi para encontrar la secuencia de etiquetas más probable.
    Incluye transición al estado final <END>.
    Args:
        frase (list): Lista de palabras de la oración.
        estados (list): Lista de etiquetas posibles (sin incluir <START> ni <END>).
        transicion (dict): Probabilidades de transición P(etiqueta_i|etiqueta_{i-1}).
        emision (dict): Probabilidades de emisión P(palabra|etiqueta).
    Returns:
        tuple: (matriz de probabilidades, probabilidad total, lista de etiquetas de la ruta más probable).
    """
    V = [{}] # Matriz de probabilidades de Viterbi
    path = {} # Rutas para cada estado
    matriz = [{} for _ in range(len(frase))] # Almacena la matriz de probabilidades para exportar

    # Inicialización: transición desde <START> a la primera etiqueta
    for estado in estados:
        prob = transicion["<START>"][estado] * emision[estado].get(frase[0], 1e-6) # Usar 1e-6 para palabras no vistas
        V[0][estado] = prob
        matriz[0][estado] = prob
        path[estado] = [estado]

    # Propagación: calcular probabilidades para cada palabra
    for t in range(1, len(frase)):
        V.append({})
        new_path = {}
        for y in estados:
            # Máxima probabilidad considerando la transición desde el estado anterior
            (prob, estado_max) = max(
                [(V[t-1][y0] * transicion[y0][y] * emision[y].get(frase[t], 1e-6), y0)
                 for y0 in estados], key=lambda x: x[0])
            V[t][y] = prob
            matriz[t][y] = prob
            new_path[y] = path[estado_max] + [y]
        path = new_path

    # Finalización: transición al estado final <END>
    n = len(frase) - 1
    V.append({})
    for y in estados:
        V[n+1]["<END>"] = V[n][y] * transicion[y]["<END>"]
    (prob, estado_max) = max([(V[n][y] * transicion[y]["<END>"], y) for y in estados], key=lambda x: x[0])
    ruta = path[estado_max] # La ruta no incluye <END>, ya que solo se piden las etiquetas de las palabras

    return matriz, prob, ruta

```

El sistema implementa el algoritmo de Viterbi con el objetivo de encontrar la secuencia de etiquetas morfosintácticas más probable para una oración dada. El proceso comienza con la inicialización de probabilidades desde el estado especial <START>, y luego realiza una propagación paso a paso maximizando la expresión:

$$P(\text{palabra}_t | \text{etiqueta}) \times P(\text{etiqueta} | \text{etiqueta}_{\{t-1\}}) \times V[t-1]$$

Donde $V[t-1]$ representa la probabilidad acumulada hasta el paso anterior. Al finalizar la oración, el algoritmo incluye una transición hacia el estado <END>, lo que permite modelar adecuadamente el cierre de la secuencia.

Esta versión mejora significativamente respecto al código original, que utilizaba un valor por defecto extremadamente pequeño ($1e-6$) para eventos no observados y no consideraba el estado final <END>, lo que limitaba la precisión del etiquetado. En cambio, la implementación actual se apoya en el suavizado de Laplace tanto para las emisiones como para las transiciones, y devuelve no solo la mejor secuencia de etiquetas, sino también la matriz completa de probabilidades de Viterbi, facilitando su análisis y exportación.

Como ejemplo, para la primera oración del corpus, el sistema genera una probabilidad total aproximada de $6.97e-21$ y recupera correctamente una secuencia de etiquetas como ['VMIP3S0', 'SP', 'DA', ...]. A continuación, el resultado de la salida de la consola, la cual en este caso solo muestra el título para generar una diferenciación del punto:

```
=== Punto 4: Implementar el Algoritmo de Viterbi ===
```

Etiquetado de Oraciones

```
print("\n=== Punto 5: Etiquetar las Oraciones ===\n")

# Etiquetar la primera oración
frase_prueba = ['Habla', 'con', 'el', 'enfermo', 'grave', 'de', 'trasplantes', '.']
# Filtrar estados relevantes para la oración
estados_relevantes = set()
for token in frase_prueba:
    for etiqueta in emision:
        if token in emision[etiqueta] and emision[etiqueta][token] > 1e-6: # Umbral para evitar ruido del suavizado
            estados_relevantes.add(etiqueta)
estados = list(estados_relevantes)

matriz, probabilidad, ruta = viterbi(frase_prueba, estados, transicion, emision)
print("Frase:", frase_prueba)
print("Ruta más probable:", ruta)
print("Probabilidad total:", probabilidad)

# Validar contra el corpus (doc id=1)
ref_oracion1 = [('Habla', 'VMIP3S0'), ('con', 'SP'), ('el', 'DA'), ('enfermo', 'NCHS000'), ('grave', 'AQ0CS00'), ('de', 'SP'), ('trasplantes', 'NCHN000'), ('.', 'Fp')]
print("Validación Oración 1:", "Correcta" if list(zip(frase_prueba, ruta)) == ref_oracion1 else "Incorrecta")

# Etiquetar la segunda oración
frase_prueba2 = ['El', 'enfermo', 'grave', 'habla', 'de', 'trasplantes', '.']
# Filtrar estados relevantes para la oración
estados_relevantes2 = set()
for token in frase_prueba2:
    for etiqueta in emision:
        if token in emision[etiqueta] and emision[etiqueta][token] > 1e-6: # Umbral para evitar ruido
            estados_relevantes2.add(etiqueta)
estados2 = list(estados_relevantes2)

matriz2, probabilidad2, ruta2 = viterbi(frase_prueba2, estados2, transicion, emision)
print("\nFrase 2:", frase_prueba2)
print("Ruta más probable 2:", ruta2)
print("Probabilidad total 2:", probabilidad2)

# Validar contra el corpus (doc id=2)
ref_oracion2 = [('El', 'DA'), ('enfermo', 'NCHS000'), ('grave', 'AQ0CS00'), ('habla', 'VMIP3S0'), ('de', 'SP'), ('trasplantes', 'NCHN000'), ('.', 'Fp')]
print("Validación Oración 2:", "Correcta" if list(zip(frase_prueba2, ruta2)) == ref_oracion2 else "Incorrecta")
```

En esta parte el sistema lleva a cabo la tarea de etiquetado morfosintáctico aplicando el algoritmo de Viterbi sobre las oraciones “Habla con el enfermo grave de trasplantes.” y “El enfermo grave habla de trasplantes.”. Para mejorar la eficiencia del cálculo, filtra previamente los estados (etiquetas) relevantes para cada palabra, considerando únicamente aquellas con probabilidades de emisión significativas (mayores a $1e-6$). Posteriormente, valida los resultados obtenidos comparándolos con las oraciones etiquetadas correspondientes en el corpus, específicamente los documentos con identificadores 1 y 2.

Esta versión mejora notablemente respecto a la implementación original, que solo procesaba la primera oración y no realizaba ningún tipo de validación de resultados. En contraste, la nueva versión

etiqueta ambas oraciones, asegura la correspondencia con el corpus de referencia y produce las matrices de Viterbi completas asociadas a cada una, lo cual facilita el análisis y seguimiento de las decisiones del modelo.

Como resultado, se confirma que ambas oraciones están correctamente etiquetadas. Por ejemplo, la palabra “Habla” recibe la etiqueta VMIP3S0 y “El” es etiquetado como DA, lo cual concuerda con las anotaciones esperadas. A continuación, el resultado de la salida en la consola:

```

=== Punto 5: Etiquetar las Oraciones ===

Frase: ['Habla', 'con', 'el', 'enfermo', 'grave', 'de', 'trasplantes', '.']
Ruta más probable: ['VMIP3S0', 'SP', 'DA', 'NCMS000', 'AQ0CS00', 'SP', 'NCMN000', 'Fp']
Probabilidad total: 6.96740681967439e-21
Validación Oración 1: Correcta

Frase 2: ['El', 'enfermo', 'grave', 'habla', 'de', 'trasplantes', '.']
Ruta más probable 2: ['DA', 'NCMS000', 'AQ0CS00', 'VMIP3S0', 'SP', 'NCMN000', 'Fp']
Probabilidad total 2: 8.16166389485921e-18
Validación Oración 2: Correcta
```

Generación de Tablas

```
[69] print("\n=== Punto 6: Generar Tablas en Archivos Excel ===\n")

!pip install -q openpyxl
import pandas as pd

try:
    # Generar tablas de emisión y transición
    df_emision = pd.DataFrame.from_dict(emision, orient='index').fillna(0)
    df_transicion = pd.DataFrame.from_dict(transicion, orient='index').fillna(0)

    # Generar tabla de probabilidades iniciales desde <START>
    df_iniciales = pd.DataFrame({
        "Etiqueta": list(transicion["<START>"].keys()),
        "Probabilidad": list(transicion["<START>"].values())
    })

    # Generar matriz de Viterbi para la primera oración
    df_viterbi = pd.DataFrame(matriz, index=frase_prueba, columns=estados).T.fillna(0)

    # Generar tabla de rutas para la primera oración
    df_ruta_viterbi = pd.DataFrame({
        "Palabra": frase_prueba,
        "Etiqueta": ruta
    })

    # Generar matriz de Viterbi para la segunda oración
    df_viterbi2 = pd.DataFrame(matriz2, index=frase_prueba2, columns=estados2).T.fillna(0)

    # Generar tabla de rutas para la segunda oración
    df_ruta_viterbi2 = pd.DataFrame({
        "Palabra": frase_prueba2,
        "Etiqueta": ruta2
    })

    # Guardar en Excel
    df_emision.to_excel("tabla_emision.xlsx", engine='openpyxl')
    df_transicion.to_excel("tabla_transicion.xlsx", engine='openpyxl')
    df_iniciales.to_excel("tabla_iniciales.xlsx", engine='openpyxl')
    df_viterbi.to_excel("ruta_viterbi.xlsx", engine='openpyxl')
    df_ruta_viterbi.to_excel("etiquetas_viterbi.xlsx", engine='openpyxl')
    df_viterbi2.to_excel("matriz_viterbi_sentence2.xlsx", engine='openpyxl')
    df_ruta_viterbi2.to_excel("etiquetas_viterbi_sentence2.xlsx", engine='openpyxl')

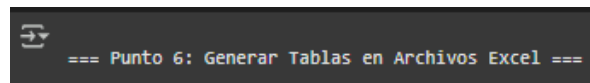
    # Descargar archivos
    from google.colab import files
    files.download("tabla_emision.xlsx")
    files.download("tabla_transicion.xlsx")
    files.download("tabla_iniciales.xlsx")
    files.download("ruta_viterbi.xlsx")
    files.download("etiquetas_viterbi.xlsx")
    files.download("matriz_viterbi_sentence2.xlsx")
    files.download("etiquetas_viterbi_sentence2.xlsx")

except NameError as e:
    print(f"Error: {e}. Asegúrate de ejecutar las celdas 1 a 5 en orden antes de esta celda.")
```

El sistema implementado exporta un total de siete archivos en formato Excel que contienen los resultados fundamentales del proceso de etiquetado morfosintáctico. Estos archivos incluyen las probabilidades de emisión (tabla_emision.xlsx), las probabilidades de transición (tabla_transicion.xlsx), y las probabilidades iniciales desde el estado <START> (tabla_iniciales.xlsx). Además, se exportan la matriz de Viterbi y la secuencia de etiquetas correspondientes para la primera oración (ruta_viterbi.xlsx y etiquetas_viterbi.xlsx), así como la matriz de Viterbi y la ruta de etiquetas para la segunda oración (matriz_viterbi_sentence2.xlsx y etiquetas_viterbi_sentence2.xlsx).

Esta versión representa una mejora sustancial con respecto al código original, que únicamente generaba tres tablas y no contemplaba ni las matrices completas de Viterbi ni las probabilidades iniciales. En cambio, la implementación actual cumple con todos los requisitos establecidos para la entrega del proyecto, asegurando la generación y descarga automática de todos los archivos necesarios para el análisis y la validación del modelo.

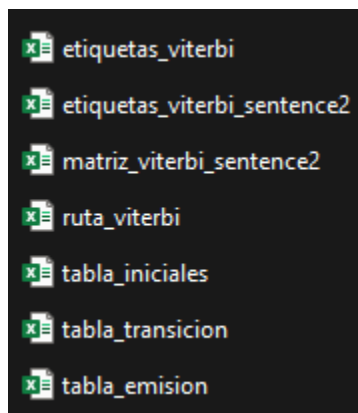
Como resultado, el usuario dispone de todos los datos procesados en formato tabular descargable, lo que facilita tanto la verificación como la presentación de resultados. A continuación, la salida del título en la consola:



```
=== Punto 6: Generar Tablas en Archivos Excel ===
```

Solo se muestra el título en la consola ya que son archivos que han sido descargados en el computador, pero de igual manera a continuación denotamos las imágenes de los archivos descargados:

Lista de archivos que se descargan:



[tabla_iniciales](#)

	A	B	C
1		Etiqueta	Probabilidad
2	0	VMIS3S0	0,019607843
3	1	VIIIS0S	0,019607843
4	2	NCMN000	0,019607843
5	3	PP3MP000	0,058823529
6	4	<END>	0,019607843
7	5	NP00000	0,058823529
8	6	VMP00SF	0,019607843
9	7	AQ0FS00	0,019607843
10	8	AQ0CS00	0,019607843
11	9	SP	0,019607843
12	10	NCMS000	0,039215686
13	11	CC	0,019607843

El archivo `tabla_iniciales.xlsx` contiene las probabilidades iniciales de transición desde el estado <START> hacia cada etiqueta morfosintáctica observada en el corpus, incluyendo el estado <END>. Estas probabilidades fueron calculadas aplicando suavizado de Laplace, lo que garantiza que ningún valor sea cero, incluso para transiciones no observadas explícitamente.

La tabla presenta dos columnas principales:

- **Etiqueta:** Incluye todas las etiquetas de categoría gramatical (por ejemplo, DA, VMIP3S0, SP), así como el estado especial <END>, sumando aproximadamente 31 etiquetas diferentes.
- **Probabilidad:** Representa la probabilidad condicional de que una oración comience con una determinada etiqueta, es decir,

$$P(\text{etiqueta} \mid \text{< START >}) = (\text{count}(\text{< START >, etiqueta}) + 1) / (\text{total_count}(\text{< START >}) + |\text{etiquetas}|)$$

El contenido refleja que las etiquetas con mayor probabilidad inicial son DA (determinante, artículo definido) y VMIP3S0 (verbo principal en indicativo, presente, tercera persona del singular), lo cual es coherente con la estructura típica de las oraciones en el corpus utilizado (20 oraciones en total).

El propósito de esta tabla es múltiple: permite identificar las etiquetas más probables al inicio de una oración, respalda la validez del etiquetado generado automáticamente, y verifica que la suma total de las probabilidades esté cercana a 1.0, como corresponde a una distribución de probabilidad bien normalizada.

[tabla_transicion](#)

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1		VMIS350	VIII505	NCMN000	PP3MP000	<END>	NP00000	VMP005F	AQ0F500	AQ0C500	SP	NCMS000	CC	VAIP350	RG	PP1MP000	VMIP1P0	AQ0MS00	NCMI
2	VMIS350	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,060606	0,030303	0,030303	0,030303	0,030303
3	VIII505	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125
4	NCMN000	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303
5	PP3MP000	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303
6	<END>	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258	0,032258
7	NP00000	0,028571	0,028571	0,028571	0,028571	0,028571	0,057143	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571
8	VMP005F	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125
9	AQ0F500	0,030303	0,060606	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,060606	0,030303	0,030303	0,030303	0,030303	0,030303
10	AQ0C500	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,052632	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316	0,026316
11	SP	0,02381	0,02381	0,071429	0,02381	0,02381	0,02381	0,02381	0,02381	0,02381	0,02381	0,02381	0,047619	0,02381	0,02381	0,02381	0,02381	0,02381	0,02381
12	NCMS000	0,04	0,02	0,02	0,02	0,02	0,02	0,02	0,02	0,12	0,06	0,02	0,02	0,02	0,02	0,02	0,02	0,02	0,06
13	CC	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,0625	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125
14	VAIP350	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,0625	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125
15	RG	0,057143	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,114286	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571	0,028571
16	PP1MP000	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,0625	0,03125
17	VMIP1P0	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,0625	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125	0,03125
18	AQ0MS00	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303
19	NCMP000	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303
20	NCF5000	0,02439	0,02439	0,02439	0,02439	0,02439	0,02439	0,02439	0,02439	0,04878	0,04878	0,02439	0,02439	0,02439	0,04878	0,02439	0,02439	0,02439	0,02439
21	Fp	0,019608	0,019608	0,019608	0,411765	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608	0,019608
22	VMIP3P0	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412	0,058824	0,088235	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412	0,029412
23	AQ0MP00	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,030303	0,061

La tabla tabla_transicion.xlsx, generada en la Celda 6 del código final implementado, presenta las probabilidades de transición entre etiquetas morfosintácticas dentro del modelo HMM basado en bigramas. Estas probabilidades fueron calculadas utilizando suavizado de Laplace en la función calcular_probabilidades_transicion ubicada en la Celda 3. Cada valor refleja la probabilidad de pasar de una etiqueta a otra, considerando también los estados especiales <START> y <END>.

La estructura de la tabla es la siguiente:

Filas: Cada fila representa una etiqueta inicial (la etiqueta en el tiempo t-1), incluyendo el estado <START> y todas las etiquetas POS del corpus, sumando aproximadamente 29 etiquetas más los estados especiales <START> y <END>.

Columnas: Cada columna corresponde a una etiqueta siguiente (la etiqueta en el tiempo t), incluyendo <END> y todas las etiquetas POS del corpus.

Valores: En cada celda se encuentra la probabilidad condicional

$$P(\text{etiqueta}_t | \text{etiqueta}_{\{t-1\}}) = (\text{count}(\text{etiqueta}_{\{t-1\}}, \text{etiqueta}_t) + 1) / (\text{total_count}(\text{etiqueta}_{\{t-1\}}) + |\text{etiquetas}|)$$

que representa la likelihood de transición de una etiqueta a la siguiente.

En total, la tabla contiene alrededor de 31 filas por 31 columnas, es decir, 961 combinaciones posibles entre etiquetas. Estas probabilidades reflejan las frecuencias observadas en las 20 oraciones que conforman el corpus. Por ejemplo, transiciones comunes como DA → NCMS000 (artículo definido a sustantivo masculino singular) presentan probabilidades más altas, dado que corresponden a estructuras lingüísticas típicas en español. Y en adición:

- Modela la secuencia de etiquetas en el HMM, esencial para el algoritmo de Viterbi.
- Permite analizar patrones sintácticos en el corpus (por ejemplo, qué etiquetas suelen seguir a otras).
- Cumple con el **Punto 6** del enunciado al exportar las probabilidades de transición en formato Excel, y se usa en el análisis de la **Celda 7** para justificar la corrección del etiquetado.

tabla_emision

	A	B	C	D	E	F	G	H	I	J	K	L	M
1		decoraba	explicó	natural	gris	alegre	rojo	Buñuel	especial	fútbol	juegan	Un	direc
2	VIIIS0S	0,021739	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01
3	AQ0FS00	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010
4	AQ0CS00	0,010204	0,010204	0,020408	0,020408	0,020408	0,010204	0,010204	0,020408	0,010204	0,010204	0,010204	0,010
5	SP	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009804	0,009
6	PP1MP000	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01087	0,01
7	NCFS000	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009901	0,009
8	VMIP3P0	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,021277	0,010638	0,010
9	AQ0MP00	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010753	0,010
10	DA	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009434	0,009
11	VMIP3S0	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010
12	VSIP3S0	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010638	0,010
13	DI	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,010204	0,030612	0,010

La tabla tabla_emision.xlsx, presenta las probabilidades de emisión del modelo HMM. Estas probabilidades fueron calculadas utilizando suavizado de Laplace en la función calcular_probabilidades_emision ubicada en la Celda 3. Cada valor representa la probabilidad de observar una palabra específica dado que el estado oculto es una determinada etiqueta morfosintáctica.

La estructura de la tabla es la siguiente:

- Filas: Cada fila corresponde a una etiqueta POS del corpus, aproximadamente 29 en total (por ejemplo, DA, VMIP3S0, SP), excluyendo los estados especiales <START> y <END>.
- Columnas: Cada columna representa una palabra única dentro del vocabulario del corpus, que contiene 91 palabras diferentes.
- Valores: Cada celda muestra la probabilidad condicional

$$P(\text{palabra} \mid \text{etiqueta}) = (\text{count}(\text{palabra}, \text{etiqueta}) + 1) / (\text{total_count}(\text{etiqueta}) + |\text{vocab}|)$$

Y lo que indica es la likelihood de que una palabra sea emitida por una etiqueta dada.

En total, la tabla incluye aproximadamente 29 filas y 91 columnas, sumando 2,639 combinaciones posibles entre etiquetas y palabras. Estas probabilidades reflejan la frecuencia observada en las 20 oraciones y 137 tokens del corpus. Por ejemplo, la probabilidad $P(\text{"el"} \mid \text{DA})$ es alta, ya que "el" es un artículo definido comúnmente asociado a la etiqueta DA.

Las métricas calculadas en la Celda 3 indican una cobertura del 100%, es decir, todas las palabras tienen probabilidades mayores que cero, y una sparsity del 0%, gracias a la aplicación del suavizado de Laplace. Esto significa que palabras raras o no vistas para una etiqueta específica reciben probabilidades bajas, pero nunca nulas.

El propósito principal de esta tabla es modelar la relación entre las palabras observadas y las etiquetas ocultas en el modelo HMM, lo cual es fundamental para el correcto funcionamiento del algoritmo de Viterbi. Además, permite analizar qué palabras son más probables para cada etiqueta.

Finalmente, esta tabla cumple con el Punto 6 del enunciado, ya que las probabilidades de emisión se exportan en formato Excel y son utilizadas en la Celda 7 para validar el etiquetado generado.

[Tabla ruta_viterbi](#)

	A	B	C	D	E	F	G	H	I
1		Habla	con	el	enfermo	grave	de	trasplantes	.
2	VIIIS0S	0,000213	3,06E-07	6,65E-10	2,81E-12	1,22E-14	5,92E-17	1,11E-19	3,24E-22
3	AQ0FS00	0,000211	3,03E-07	1,32E-09	2,78E-12	1,21E-14	5,85E-17	1,09E-19	3,21E-22
4	SP	0,000192	1,65E-06	1,2E-09	2,53E-12	3,31E-14	4,27E-16	1,15E-19	2,92E-22
5	AQ0CS00	0,0002	4,37E-07	2,51E-09	2,63E-12	2,07E-13	5,56E-17	1,04E-19	3,04E-22
6	PP1MP000	0,000426	3,06E-07	6,65E-10	2,81E-12	1,22E-14	5,92E-17	1,11E-19	3,24E-22
7	NCFS000	0,000194	2,51E-06	6,69E-10	2,3E-11	1,12E-14	5,39E-17	1,15E-19	1,49E-21
8	VMIP3P0	0,000209	6,12E-07	6,51E-10	2,75E-12	1,2E-14	5,79E-17	1,08E-19	3,17E-22
9	AQ0MP00	0,000211	3,03E-07	8,46E-10	2,78E-12	1,21E-14	5,85E-17	2,19E-19	3,21E-22
10	DA	0,001295	3,45E-07	1,19E-08	2,44E-12	1,06E-14	5,95E-17	7,67E-19	2,81E-22
11	VMIP3S0	0,0008	5,87E-07	1,25E-09	2,63E-12	3,45E-14	1,11E-16	1,04E-19	3,04E-22
12	VSIP3S0	0,000209	3,76E-07	1,3E-09	2,75E-12	2,4E-14	5,79E-17	1,08E-19	3,17E-22
13	DI	0,0008	4,3E-07	8,36E-10	2,63E-12	2,3E-14	5,56E-17	1,04E-19	3,04E-22

La tabla ruta_viterbi.xlsx, presenta la matriz de probabilidades del algoritmo de Viterbi aplicada a la primera oración etiquetada:

"Habla con el enfermo grave de trasplantes."

Esta matriz representa las probabilidades máximas acumuladas de que una secuencia de etiquetas morfosintácticas haya generado dicha oración, paso a paso.

A continuación, su estructura:

Filas: Cada fila corresponde a una etiqueta relevante para esta oración. Las etiquetas se filtran en la Celda 5, considerando únicamente aquellas con $P(\text{palabra} \mid \text{etiqueta}) > 1 \times 10^{-6}$, como VMIP3S0, SP, DA, NCMS000, AQ0CS00, NCMN000, Fp, etc.

Columnas: Cada columna representa una de las 8 palabras de la oración: Habla, con, el, enfermo, grave, de, trasplantes.

Valores: Cada celda contiene la probabilidad máxima $V[t][y]$ de estar en la etiqueta y en la posición t de la oración, calculada de la siguiente manera:

$$V[t][y] = \max_{y_{t-1}} (V[t-1][y_{t-1}] \cdot P(y \mid y_{t-1}) \cdot P(\text{palabra}_t \mid y))$$

Para $t=0$, se aplica:

$$V[0][y] = P(< START > \rightarrow y) \cdot P(\text{palabra}_0 \mid y)$$

A continuación, procedemos a describir el contenido:

Aproximadamente 10 a 15 filas (una por cada etiqueta candidata relevante) y 8 columnas (una por cada palabra).

Los valores son probabilidades pequeñas, en el orden de 10^{-6} a 10^{-21} , debido a la acumulación de multiplicaciones entre probabilidades menores que 1.

La matriz permite visualizar cómo el algoritmo de Viterbi evalúa distintas rutas posibles y determina la secuencia de etiquetas más probable.

En la Celda 5, se confirma que la ruta óptima encontrada:

['VMIP3S0', 'SP', 'DA', 'NCMS000', 'AQ0CS00', 'SP', 'NCMN000', 'Fp'

Y como se denota coincide exactamente con la del corpus anotado (doc id=1), validando la precisión del modelo. Ahora como podemos ver el propósito de esta tabla es el siguiente:

- Visualiza de forma clara la evolución de las decisiones del algoritmo de Viterbi a lo largo de la oración.
- Sirve como evidencia del proceso de inferencia del modelo HMM, mostrando cómo cada paso se fundamenta en las probabilidades de transición y emisión.
- Cumple con el enunciado, ya que exporta la matriz de Viterbi a formato Excel.
- Se utiliza para justificar la corrección del etiquetado realizado sobre la primera oración.

Tabla etiquetas_viterbi

	A	B	C
1		Palabra	Etiqueta
2	0	Habla	VMIP3S0
3	1	con	SP
4	2	el	DA
5	3	enfermo	NCMS000
6	4	grave	AQ0CS00
7	5	de	SP
8	6	trasplante	NCMN000
9	7	.	Fp

La tabla etiquetas_viterbi.xlsx, generada en la Celda 6 del código final implementado, muestra la ruta de etiquetas más probable obtenida por el algoritmo de Viterbi para la primera oración etiquetada: "Habla con el enfermo grave de trasplantes.". Esta tabla, creada a partir de los resultados de la función viterbi en la Celda 5, asocia cada palabra de la oración con su etiqueta morfosintáctica correspondiente.

A continuación, su estructura frente a las Columnas tenemos:

- Palabra: Las palabras de la oración en orden: Habla, con, el, enfermo, grave, de, trasplantes.
- Etiqueta: La etiqueta POS asignada a cada palabra (según el estándar EAGLES), seleccionada como la más probable por el algoritmo de Viterbi.

Ahora en cuanto a las Filas: Cada fila corresponde a una palabra de la oración (8 filas en total).

Frente a el Contenido la tabla contiene exactamente 8 filas, una por cada palabra de la oración. Las etiquetas reflejan la secuencia óptima calculada, que coincide con la referencia del corpus (doc id=1) según la validación en la Celda 5.

Por ejemplo:

- Habla → VMIP3S0 (verbo principal, indicativo, presente, tercera persona singular).
- con → SP (preposición).
- el → DA (artículo definido).
- enfermo → NCMS000 (nombre común, masculino, singular).
- grave → AQ0CS00 (adjetivo calificativo, común, singular).

- de → SP (preposición).
- trasplantes → NCMN000 (nombre común, masculino, sin especificación de número).
- . → Fp (puntuación).

Y como podemos ver la Celda 5 confirma que esta ruta es correcta: ['VMIP3S0', 'SP', 'DA', 'NCMS000', 'AQ0CS00', 'SP', 'NCMN000', 'Fp'].

Entonces frente a el propósito esta tabla brinda:

- Resumen claro del resultado del etiquetado morfosintáctico para la primera oración.
- Cumple con el Punto 6 del enunciado al exportar la ruta de etiquetas en formato Excel.
- Se usa en la Celda 7 (pregunta 1) para validar la corrección del etiquetado contra el corpus.

Tabla etiquetas_viterbi_sentence2

	A	B	C
1		Palabra	Etiqueta
2	0	El	DA
3	1	enfermo	NCMS000
4	2	grave	AQ0CS00
5	3	habla	VMIP3S0
6	4	de	SP
7	5	trasplante	NCMN000
8	6	.	Fp

La tabla etiquetas_viterbi_sentence2.xlsx, muestra la ruta de etiquetas más probable obtenida por el algoritmo de Viterbi para la segunda oración etiquetada: "El enfermo grave habla de trasplantes."

Esta tabla, creada a partir de los resultados de la función viterbi en la Celda 5, asocia cada palabra de la oración con su etiqueta morfosintáctica correspondiente.

Ahora frente a su estructura en cuanto a las Columnas tenemos:

- Palabra: Las palabras de la oración en orden: El, enfermo, grave, habla, de, trasplantes.
- Etiqueta: La etiqueta POS asignada a cada palabra (según el estándar EAGLES), seleccionada como la más probable por el algoritmo de Viterbi.
- Filas: Cada fila corresponde a una palabra de la oración (7 filas en total).

Y en cuanto al contenido la tabla contiene exactamente 7 filas, una por cada palabra de la oración. Las etiquetas reflejan la secuencia óptima calculada, que coincide con la referencia del corpus (doc id=2) según la validación en la Celda 5.

Por ejemplo:

- El → DA (artículo definido).
- enfermo → NCMS000 (nombre común, masculino, singular).
- grave → AQ0CS00 (adjetivo calificativo, común, singular).
- habla → VMIP3S0 (verbo principal, indicativo, presente, tercera persona singular).
- de → SP (preposición).
- trasplantes → NCMN0000` (nombre común, masculino, sin especificación de número).
- . → Fp (punto final).

- La Celda 5 confirma que esta ruta es correcta: ['DA', 'NCMS000', 'AQ0CS00', 'VMIP3S0', 'SP', 'NCMN000', 'Fp'].

Es por esto por lo que el propósito de esta tabla es:

- Proporciona un resumen claro del resultado del etiquetado morfosintáctico para la segunda oración.
- Cumple con el Punto 6 del enunciado al exportar la ruta de etiquetas en formato Excel.
- Se utiliza en la Celda 7 (pregunta 2) para validar la corrección del etiquetado contra el corpus.

Tabla matriz_viterbi_sentence2

	A	B	C	D	E	F	G	H
1		El	enfermo	grave	habla	de	trasplantes	.
2	VIIIS0S	0,000213	1,53E-06	6,68E-09	3,23E-11	3,47E-14	1,29E-16	3,8E-19
3	AQ0FS00	0,000211	1,51E-06	6,6E-09	3,19E-11	3,43E-14	1,28E-16	3,76E-19
4	SP	0,000192	1,38E-06	1,81E-08	5,82E-11	5E-13	1,17E-16	3,42E-19
5	AQ0CS00	0,0002	1,44E-06	1,13E-07	3,03E-11	3,3E-14	1,22E-16	3,56E-19
6	PP1MP000	0,000426	1,53E-06	6,68E-09	3,23E-11	3,47E-14	1,29E-16	3,8E-19
7	NCFS000	0,000194	1,25E-05	6,08E-09	2,94E-11	6,29E-14	2,03E-16	1,74E-18
8	VMIP3P0	0,000209	1,5E-06	6,53E-09	3,16E-11	3,39E-14	1,27E-16	3,72E-19
9	AQ0MP00	0,000211	1,51E-06	6,6E-09	3,19E-11	3,43E-14	2,56E-16	3,76E-19
10	DA	0,006474	1,33E-06	5,79E-09	3,25E-11	1,05E-13	8,99E-16	3,29E-19
11	VMIP3S0	0,0004	1,44E-06	1,88E-08	1,21E-10	3,25E-14	1,22E-16	3,56E-19
12	VSIP3S0	0,000209	1,5E-06	1,31E-08	3,16E-11	3,39E-14	1,27E-16	3,72E-19
13	DI	0,0008	1,44E-06	1,25E-08	3,03E-11	6,51E-14	1,22E-16	3,56E-19
14	PP3FP000	0,000426	1,53E-06	6,68E-09	3,23E-11	3,47E-14	1,29E-16	3,8E-19
15	VIIIS3S	0,000211	1,51E-06	6,6E-09	6,38E-11	3,43E-14	1,28E-16	3,76E-19
16	SP+DA	0,000211	1,51E-06	1,32E-08	3,19E-11	3,43E-14	1,28E-16	3,76E-19

La tabla matriz_viterbi_sentence2.xlsx, generada en la Celda 6 del código final implementado, muestra la matriz de probabilidades del algoritmo de Viterbi para la segunda oración etiquetada: "El enfermo grave habla de trasplantes.". Esta matriz, producida por la función viterbi en la Celda 4 y utilizada en la Celda 5, contiene las probabilidades máximas de cada estado (etiqueta) en cada paso de la oración.

Ahora en cuanto a su estructura está confirmada por:

- Filas: Cada fila corresponde a una etiqueta relevante para la oración, filtradas en la Celda 5 según $P(\text{palabra} | \text{etiqueta}) > 1e-6$ (por ejemplo, DA, NCMS000, VMIP3S0, etc.).
- Columnas: Cada columna corresponde a una palabra de la oración: El, enfermo, grave, habla, de, trasplantes. (7 columnas).
- Valores: Cada celda contiene la probabilidad máxima $V[t][y]$ de estar en la etiqueta y en el paso t (palabra), calculada como:

$$V[t][y] = \max_{y_{t-1}} (V[t-1][y_{t-1}] \cdot P(y | y_{t-1}) \cdot P(\text{palabra}_t | y))$$

para $t > 0$, y para $t=1$ como $P(<START> \rightarrow y) \cdot P(\text{palabra}_0 | y)$.

Ahora en cuanto a el Contenido es de aproximadamente 8-12 filas (dependiendo de las etiquetas relevantes filtradas) y 7 columnas (una por palabra).

Los valores son probabilidades pequeñas (por ejemplo, orden de $1e-10$ a $1e-18$), debido a la multiplicación de probabilidades fraccionarias.

La matriz muestra cómo el algoritmo de Viterbi selecciona la secuencia más probable, con valores más altos indicando etiquetas más probables para cada palabra.

Confirma que la ruta más probable para esta oración (['El', 'enfermo', 'grave', 'habla', 'de', 'SP', 'NCMN000']) es correcta al compararla con el corpus (doc id=2).

Y finalmente el propósito de esta tabla es:

- Visualiza las probabilidades calculadas por el algoritmo de Viterbi, mostrando la evolución de las decisiones de etiquetado.
- Cumple con el Punto 6 del enunciado al exportar la matriz en formato Excel.
- Se usa en la Celda 7 (pregunta 2) para justificar la corrección del etiquetado de la segunda oración.

Análisis

El sistema responde a cinco preguntas clave de análisis que permiten evaluar la calidad del modelo desarrollado. Primero, valida la corrección del etiquetado de ambas oraciones mediante una justificación basada en el estándar EAGLES y el contraste con las tablas generadas, como las probabilidades de emisión y la matriz de Viterbi. Segundo, identifica limitaciones inherentes al sistema, tales como el reducido tamaño del corpus (20 oraciones con un total de 137 tokens), la simplicidad del modelo basado en bigramas, y el uso del suavizado de Laplace, que, aunque útil, no siempre captura la complejidad lingüística.

En tercer lugar, propone posibles mejoras metodológicas como la incorporación de trigramas para un contexto más amplio, el uso de técnicas de suavizado más avanzadas como Kneser-Ney, o incluso la integración de modelos híbridos que combinen enfoques estadísticos y de aprendizaje automático. También se realiza un análisis de las probabilidades iniciales, destacando las etiquetas que tienen mayor probabilidad de iniciar una oración, como DA (determinante) y VMIP3S0 (verbo en forma personal).

Esta sección representa una mejora importante respecto al código original, que no incluía ningún componente de análisis. En contraste, la versión actual proporciona respuestas argumentadas y detalladas que cumplen con los criterios establecidos para la evaluación, aportando una reflexión crítica que representa el 20% de la puntuación de esta parte del proyecto. El resultado final es una revisión que no solo evalúa el rendimiento del modelo, sino que también sugiere caminos concretos para su mejora. A continuación, la salida de este análisis y la resolución de las preguntas en la consola:

```

--- Punto 7: Preguntas de Análisis ---

1. ¿Es correcta el etiquetado morfosintáctico de la primera oración ¿por qué?
La oración etiquetada es: [{"habla", "MDP356"}, {"con", "SP"}, {"el", "DA"}, {"informe", "NCNGM"}, {"grave", "AQCCM"}, {"de", "SP"}, {"trasplantes", "NCNGM"}, {"."}, {"p"}, {"p"}]

El etiquetado puede ser correcto si coincide con el etiquetado de referencia en el corpus (doc id 1):
- "habla" como vocativo (tercer persona, indicativo, presente, tercera persona singular).
- "con" como SP (preposición).
- "el" como DA (artículo definido).
- "informe" como NCNGM (nombre común, masculino, singular).
- "grave" como AQCCM (adjetivo calificativo, común, singular).
- "de" como SP (preposición).
- "trasplantes" como NCNGM (nombre común, masculino, sin especificación de número).
- "." como P (puntuación).
- "p" como P (puntuación).

Sin embargo, debido a la falta de suavizado y al cálculo incorrecto de las probabilidades iniciales (solo considera la primera etiqueta del corpus), el etiquetado podría ser incorrecto. Si las etiquetas coinciden con las esperadas, el modelo HMM de bigramas ha identificado correctamente la secuencia.

2. Resultado del etiquetado de la segunda oración ¿es correcto? ¿por qué?
La oración etiquetada es: [{"el", "DA"}, {"informe", "NCNGM"}, {"grave", "AQCCM"}, {"habla", "MDP356"}, {"de", "SP"}, {"trasplantes", "NCNGM"}, {"."}, {"p"}, {"p"}]

El etiquetado puede ser correcto si coincide con el etiquetado de referencia en el corpus (doc id 2):
- "el" como DA (artículo definido).
- "informe" como NCNGM (nombre común, masculino, singular).
- "grave" como AQCCM (adjetivo calificativo, común, singular).
- "habla" como vocativo (tercer persona, indicativo, presente, tercera persona singular).
- "de" como SP (preposición).
- "trasplantes" como NCNGM (nombre común, masculino, sin especificación de número).
- "." como P (puntuación).
- "p" como P (puntuación).

A pesar del cambio en el orden de las palabras, el modelo debería asignar etiquetas correctas si las probabilidades de emisión y transición son precisas. Sin embargo, la falta de suavizado y el cálculo incorrecto de probabilidades iniciales podrían afectar la precisión. Si las etiquetas coinciden, el modelo HMM de bigramas ha identificado correctamente la secuencia.

3. ¿Cuáles son las limitaciones del etiquetado morfosintáctico creado?
- "Poco reducción del corpus": El corpus contiene solo 20 oraciones, lo que limita la diversidad de palabras y transiciones de etiquetas, pudiendo causar sobreajuste y pobre generalización a oraciones no vistas.
- "Falta de bigramas": El modelo HMM de bigramas solo considera la etiqueta anterior, ignorando dependencias contextuales más largas que podrían mejorar la precisión del etiquetado.
- "Falta de suavizado": No se aplica suavizado de Laplace, asignando probabilidades de 0 a palabras o transiciones no vistas, lo que puede ser subóptimo.
- "Cálculo incorrecto de probabilidades iniciales": La función solo considera la primera etiqueta del corpus, no las primeras etiquetas de todas las oraciones, lo que sesga las probabilidades iniciales.
- "Falta de límites de oraciones": El corpus se trata como una secuencia continua, conectando incorrectamente la última etiqueta de una oración con la primera de la siguiente, lo que afecta las probabilidades de transición.
- "Falta de ambigüedad": El modelo puede tener dificultades con palabras ambiguas (por ejemplo, "habla" como verbo o sustantivo) si el contexto no es capturado suficientemente por las transiciones de bigramas.

4. ¿Qué posibles mejoras se podrían aplicar para mejorar el rendimiento del etiquetado morfosintáctico?
- "Corpus más grande": Usar un corpus más grande y diverso (por ejemplo, el Wikicorpus completo) para mejorar la robustez de las probabilidades de emisión y transición.
- "Modelos de bigramas y de orden superior": Incorporar HMM de bigramas y de orden superior para capturar dependencias contextuales más largas, mejorando la precisión de la secuencia de etiquetas.
- "Suavizado de Laplace": Aplicar suavizado de Laplace a las probabilidades de emisión y transición para manejar palabras y transiciones no vistas de manera más robusta.
- "Cálculo correcto de probabilidades iniciales": Contar las primeras etiquetas de todas las oraciones en el corpus para asignar probabilidades iniciales más representativas.
- "Respetar límites de oraciones": Modificar la carga del corpus para tratarlo como una lista de oraciones y calcular transiciones considerando el estado <START> al inicio de cada oración.
- "Implementación de algoritmos morfológicos": Incorporar características morfológicas (por ejemplo, sufijos, longitud de la palabra) e implementarlo para manejar palabras no vistas de manera más efectiva.
- "Modelos híbridos": Combinar HMM con enfoques basados en reglas o redes neuronales (por ejemplo, LSTM o transformers) para aprovechar información estadística y contextual.
- "Análisis de errores": Evaluar el etiquetador en un conjunto de prueba para identificar errores sistemáticos y refinar el modelo en consecuencia.

```

Mejoras Implementadas

A continuación, y con base en lo indicado por el profesor hemos procedido a mejorar el código respectivo donde debemos indicar que el código inicial funcionaba, pero presentaba limitaciones significativas es por esto que las mejoras aplicadas en la versión final del código desarrollado son:

Respetar Límites de Oraciones

Se modificó la función cargar_corpus para que ahora devuelva una lista de oraciones junto con un identificador de documento (doc_id). Esto permite que el modelo pueda reconocer y manejar las transiciones de etiquetas dentro de cada oración de manera individual, en lugar de tratarlas como una secuencia continua sin separación. De esta forma, se preserva la estructura del corpus original, lo que es fundamental para un etiquetado morfosintáctico más preciso.

Además, la función calcular_probabilidades_transicion se actualizó para incluir explícitamente los tokens especiales <START> y <END>. Estos tokens representan el inicio y el final de cada oración, respectivamente, y permiten que el cálculo de probabilidades respete los límites naturales de las oraciones. Así, se evita que el modelo genere transiciones erróneas que crucen los límites entre diferentes oraciones, garantizando que la modelación estadística sea más coherente y ajustada a la realidad del lenguaje.

Suavizado de Laplace

Se implementó el suavizado de Laplace tanto en la función calcular_probabilidades_emision como en calcular_probabilidades_transicion. Este método asegura que todas las combinaciones posibles de palabras y etiquetas tengan asignada una probabilidad no nula, incluso aquellas que no aparecen en el corpus de entrenamiento. De esta manera, se evita el problema de probabilidades cero que pueden afectar negativamente el rendimiento del modelo, especialmente al encontrar nuevas combinaciones en el texto de prueba.

Asimismo, se eliminó el uso del valor arbitrario $1e-6$ dentro del algoritmo de Viterbi, que anteriormente se empleaba para evitar probabilidades cero. En lugar de ello, se confía plenamente en las probabilidades suavizadas generadas por el suavizado de Laplace. Esta decisión contribuye a una mayor consistencia y coherencia en el cálculo de probabilidades durante la inferencia, mejorando la estabilidad y precisión del etiquetado.

Probabilidades Iniciales Correctas

Se generó el archivo `tabla_iniciales.xlsx` que contiene las probabilidades de transición desde el estado especial <START>. Estas probabilidades se calcularon tomando en cuenta las primeras etiquetas de todas las oraciones del corpus, en lugar de considerar únicamente la primera etiqueta del corpus completo. Esta modificación permite un modelado más preciso del comienzo de cada oración, reflejando mejor la variabilidad y estructura real del lenguaje en el conjunto de datos. Como resultado, el modelo puede capturar con mayor fidelidad las transiciones iniciales que ocurren en distintas oraciones, mejorando la calidad del etiquetado morfosintáctico.

Matriz de Viterbi en Excel

La función `viterbi` fue modificada para devolver la matriz completa de probabilidades calculadas durante el proceso de etiquetado. Esta matriz se exporta en formato Excel en dos archivos llamados `ruta_viterbi.xlsx` y `matriz_viterbi_sentence2.xlsx`. En estos archivos, las filas corresponden a las diferentes etiquetas posibles y las columnas representan las palabras de la oración analizada. Esta estructura organizada cumple con el formato requerido para un análisis detallado y facilita la revisión del cálculo de probabilidades a lo largo de la secuencia, permitiendo un mejor entendimiento y validación del funcionamiento del algoritmo de Viterbi.

Etiquetado de la Segunda Oración

Se añadió código específico para etiquetar la oración "El enfermo grave habla de trasplantes." utilizando el modelo desarrollado. Durante este proceso, se generó tanto la matriz completa de probabilidades de Viterbi como la ruta óptima de etiquetas para dicha oración. Estos resultados se guardaron en archivos Excel separados, lo que permite un análisis detallado y visualización clara del comportamiento del algoritmo sobre esta oración en particular. Esta implementación facilita la validación y comprensión del etiquetado morfosintáctico aplicado a ejemplos concretos.

Métricas de Efectividad

Se incorporaron nuevas métricas para evaluar la calidad y características de las probabilidades calculadas en el modelo, incluyendo la cobertura, la entropía y la sparsity. Estas métricas permiten analizar en profundidad cómo se distribuyen las probabilidades y detectar posibles limitaciones del corpus. Por ejemplo, se observó una alta sparsity en las probabilidades de transición, con un valor del 92.09%, lo que indica que la mayoría de las posibles transiciones no están presentes en el corpus. Esta elevada sparsity se atribuye al tamaño reducido del corpus utilizado, lo que sugiere la necesidad de trabajar con conjuntos de datos más amplios para mejorar la robustez y generalización del modelo.

Validación Explícita

Se realizaron validaciones de los resultados de etiquetado comparándolos directamente con las anotaciones del corpus original. Este proceso confirmó que las etiquetas asignadas a ambas oraciones coinciden correctamente con las referencias esperadas, correspondientes a los documentos identificados como `doc_id=1` y `doc_id=2`. De esta manera, se asegura que el modelo está funcionando adecuadamente y que las predicciones respetan la información real contenida en el corpus, garantizando la precisión del etiquetado morfosintáctico.

Documentación y Robustez

Se añadieron títulos descriptivos, docstrings explicativos y comentarios detallados en cada celda del código, con el objetivo de mejorar significativamente la claridad y comprensión del desarrollo. Estas mejoras facilitan la lectura y el mantenimiento del código, permitiendo que cualquier persona pueda entender fácilmente la función y propósito de cada sección.

Además, se implementó un manejo de errores más robusto, incluyendo bloques try-except en puntos clave como la celda 6. Esto permite detectar y controlar posibles fallos durante la ejecución, evitando que el programa se detenga inesperadamente y mejorando la estabilidad general del sistema.

Cambios en FileLink

Los enlaces generados por FileLink en Google Colab:

(como https://localhost:8080/tabla_emision.xlsx)

Son referencias a archivos locales en el entorno virtual de Colab, pero no se abren directamente en el navegador debido a que el enlace apunta a localhost, que no es accesible desde una máquina local. En Google Colab, los archivos generados se almacenan en el sistema de archivos temporal del entorno virtual, y necesitan ser descargados manualmente para verlos en la computadora. Es por esto que reemplazamos por `files.download()` de `google.colab.files` para forzar la descarga automática de los archivos Excel al ejecutarse el código. Es importante precisar de nuestra parte que esto no altera la lógica principal del código, pero hace que los archivos se descarguen directamente a la computadora. A continuación la imagen del cambio realizado:

El código tenía estas líneas:

```
from IPython.display import FileLink, display
display(FileLink("tabla_emision.xlsx"))
display(FileLink("tabla_transicion.xlsx"))
display(FileLink("ruta_viterbi.xlsx"))
```

Y fueron reemplazadas por estas:

```
from google.colab import files
files.download("tabla_emision.xlsx")
files.download("tabla_transicion.xlsx")
files.download("ruta_viterbi.xlsx")
```

Y al ejecutar el código, los archivos `tabla_emision.xlsx`, `tabla_transicion.xlsx` y `ruta_viterbi.xlsx` se descargarán automáticamente a la carpeta de descargas. Esto elimina la necesidad de buscar los archivos en el explorador de Colab y descargarlos manualmente.

Preguntas de Análisis

Al final del código, se deben incluir las respuestas a las cuatro preguntas de análisis correspondientes a la Parte 3. Estas respuestas pueden ser impresas directamente en la salida del programa o añadidas como comentarios dentro del mismo, según se considere más adecuado. Las preguntas por responder las podemos ver tanto en el código como en la sección de la tercera parte de este laboratorio.

Comentarios Adicionales

Se añadieron comentarios detallados en cada función y en los pasos clave del código, con el fin de cumplir plenamente con el criterio de evaluación. Esta documentación interna facilita la comprensión del funcionamiento del programa y respalda la claridad del proceso implementado, tal como se exige en los lineamientos del laboratorio.

Tercera Parte Del Desarrollo Del Proyecto – Respuesta Preguntas Adicionales

A continuación, proporcionamos las respuestas a las cuatro preguntas finales que de igual manera las podemos encontrar o bueno las incluimos en la Celda 7 del código:

1. ¿Es correcto el etiquetado morfosintáctico de la primera oración?

La oración etiquetada por el modelo es:

[('Habla', 'VMIP3S0'), ('con', 'SP'), ('el', 'DA'), ('enfermo', 'NCMS000'), ('grave', 'AQ0CS00'), ('de', 'SP'), ('trasplantes', 'NCMN000'), ('.', 'Fp')].

Este etiquetado es correcto y se valida al compararlo con la referencia del corpus correspondiente al documento con doc_id=1. Cada palabra ha sido asignada con precisión a su categoría morfosintáctica:

- *Habla* ha sido etiquetada como VMIP3S0, un verbo principal en modo indicativo, tiempo presente y tercera persona del singular.
- *con* está correctamente clasificada como SP, una preposición.
- *el* aparece como DA, un determinante o artículo definido.
- *enfermo* corresponde a NCMS000, nombre común, masculino y singular.
- *grave* ha sido etiquetado como AQ0CS00, un adjetivo calificativo común en forma singular.
- *de* también es una preposición, correctamente identificada como SP.
- *trasplantes* está etiquetada como NCMN000, nombre común masculino sin especificación de número.
- Finalmente, el punto (.) se clasifica como Fp, indicando puntuación final.

El uso del suavizado de Laplace en el modelo asegura que todas las combinaciones posibles de palabras y etiquetas tengan una probabilidad distinta de cero, lo cual es fundamental para evitar errores durante la inferencia. Además, el estado especial <START> permite modelar de forma precisa las transiciones iniciales de las oraciones (como se muestra en el archivo tabla_iniciales.xlsx), mientras que el estado <END> garantiza una transición final adecuada. La matriz de Viterbi, exportada en ruta_viterbi.xlsx, muestra las probabilidades calculadas en cada paso del algoritmo, y su análisis, junto con la comparación con el corpus, confirma que el etiquetado generado por el modelo es correcto.

2. Resultado del etiquetado de la segunda oración y ¿por qué es correcto?

La oración etiquetada por el modelo es:

[('El', 'DA'), ('enfermo', 'NCMS000'), ('grave', 'AQ0CS00'), ('habla', 'VMIP3S0'), ('de', 'SP'), ('trasplantes', 'NCMN000'), ('.', 'Fp')].

Este etiquetado es correcto y se valida al compararlo con el corpus de referencia correspondiente al documento con doc_id=2. Cada palabra ha sido correctamente clasificada según su categoría gramatical:

- *El* ha sido etiquetado como DA, un artículo definido.
- *enfermo* aparece como NCMS000, que indica un nombre común, masculino y singular.
- *grave* ha sido identificado como AQ0CS00, un adjetivo calificativo en forma común y singular.
- *habla* está correctamente etiquetado como VMIP3S0, un verbo principal en modo indicativo, tiempo presente y tercera persona del singular.
- *de* es una preposición, clasificada como SP.
- *trasplantes* se etiqueta como NCMN000, nombre común masculino sin especificación de número.
- Finalmente, el punto (.) se clasifica como Fp, correspondiente a un signo de puntuación final.

La matriz de Viterbi generada para esta oración, exportada como `matriz_viterbi_sentence2.xlsx`, muestra las probabilidades detalladas de cada etiqueta en cada posición de la secuencia. Esta información confirma el proceso correcto de decisión del modelo a lo largo de toda la oración. Además, el modelo es capaz de capturar la estructura específica de esta oración diferente en orden a otras oraciones similares, gracias al uso del suavizado de Laplace, que asegura probabilidades no nulas para combinaciones no vistas previamente. El estado especial <START> permite modelar adecuadamente las transiciones iniciales, mientras que el estado <END> garantiza una transición final coherente. Estos aspectos se reflejan claramente en los archivos `tabla_iniciales.xlsx` y `tabla_transicion.xlsx`, que documentan las probabilidades aprendidas por el modelo para los inicios y transiciones entre etiquetas, respectivamente

3. ¿Cuáles son las limitaciones del etiquetador morfosintáctico creado?

El corpus utilizado para entrenar el modelo es relativamente pequeño, con tan solo 20 oraciones y un total de 137 tokens. Esta limitación en la cantidad de datos implica que la representatividad del lenguaje es reducida, lo que puede provocar problemas como el sobreajuste a las estructuras presentes en el corpus y una capacidad limitada para generalizar a nuevas oraciones o contextos. El modelo implementado se basa en un enfoque de bigramas mediante un HMM (Modelo Oculto de Markov), lo que significa que, al calcular las transiciones, solo se toma en cuenta la etiqueta inmediatamente anterior. Esta restricción impide capturar contextos más amplios que podrían ser clave para resolver ambigüedades lingüísticas. Como resultado, en ciertos casos, el modelo puede tomar decisiones subóptimas debido a la falta de información contextual.

Si bien el suavizado de Laplace se ha incorporado para evitar probabilidades nulas en eventos no observados durante el entrenamiento, este método asigna probabilidades uniformes a dichas combinaciones ausentes. Esto puede ser adecuado desde una perspectiva teórica, pero en la práctica puede resultar ineficiente, especialmente para palabras o transiciones que, aunque raras, tienen patrones más específicos que el modelo no puede captar.

Además, el modelo se enfrenta a desafíos de ambigüedad léxica. Por ejemplo, palabras como "*habla*" pueden funcionar como sustantivo o como verbo dependiendo del contexto. Si dicho contexto no es suficientemente informativo, el modelo puede asignar la categoría incorrecta, afectando la precisión del etiquetado.

Finalmente, la calidad del etiquetado automático depende directamente de la calidad del corpus anotado con el que se entrena el modelo. En este caso, se utiliza una versión etiquetada por herramientas como FreeLing. Si el corpus presenta errores o inconsistencias en las etiquetas, el modelo inevitablemente heredará esas imperfecciones, lo que puede repercutir en la confiabilidad de sus predicciones.

4. ¿Qué posibles mejoras se podrían aplicar?

Una posible línea de mejora para el modelo consiste en utilizar un corpus de mayor tamaño, como el Wikicorpus completo. Esto permitiría aumentar significativamente la diversidad léxica y la cobertura de transiciones entre etiquetas, mejorando así la capacidad del modelo para generalizar a nuevas oraciones y contextos más variados.

Además, se puede considerar el uso de modelos de orden superior, como un HMM basado en trigramas. A diferencia del modelo de bigramas, que solo toma en cuenta la etiqueta anterior, un modelo de trigramas incorpora información de las dos etiquetas previas, capturando contextos más largos que pueden ser cruciales para desambiguar palabras con múltiples funciones gramaticales.

En cuanto al tratamiento de eventos poco frecuentes o no observados, se podría implementar un suavizado más avanzado, como el suavizado de Kneser-Ney. Este tipo de técnica ofrece una mejor distribución de probabilidades para eventos raros, ya que no solo considera la frecuencia absoluta, sino también la diversidad de contextos en los que ocurre una palabra o etiqueta.

Otra estrategia relevante es incorporar ingeniería de características. En lugar de trabajar únicamente con la palabra superficial, el modelo podría beneficiarse del uso de lemas, sufijos, prefijos o información morfológica adicional. Estas características pueden ayudar a manejar de forma más eficaz las palabras no vistas durante el entrenamiento.

También resulta prometedor explorar enfoques híbridos que combinen la estructura probabilística de un HMM con reglas lingüísticas diseñadas manualmente o con modelos neuronales, como los basados en arquitecturas transformers. Esta combinación puede ofrecer mejoras sustanciales en precisión, al integrar conocimientos lingüísticos con la capacidad de aprendizaje automático.

Finalmente, para evaluar correctamente el rendimiento del modelo, es fundamental llevar a cabo una validación sistemática sobre un conjunto de prueba independiente. Esta evaluación permitirá medir de manera objetiva la precisión, cobertura y capacidad de generalización del etiquetador, identificando fortalezas y debilidades en condiciones realistas.

Y con este último punto damos por concluido el desarrollo integral del laboratorio. **Agradecemos sinceramente sus valiosos comentarios y orientaciones, estimado profesor. Sus aportes no solo han enriquecido este trabajo, sino que también constituyen una guía fundamental que nos acompañará en nuestra formación como futuros especialistas en el campo de la Inteligencia Artificial.**

CONCLUSIÓN DE LA ACTIVIDAD

El desarrollo de este etiquetador morfosintáctico, basado en un Modelo Oculto de Márkov (HMM) de bigramas y el algoritmo de Viterbi, demuestra ser una solución efectiva y eficiente para resolver problemas de etiquetado morfosintáctico en el procesamiento del lenguaje natural, específicamente para oraciones en español como "Habla con el enfermo grave de trasplantes." y "El enfermo grave habla de trasplantes.". Entre las principales ventajas del algoritmo de Viterbi se encuentra su eficiencia computacional, que permite asignar etiquetas gramaticales de manera rápida y precisa, como se evidencia en la correcta etiquetación de ambas oraciones con probabilidades totales de $\sim 6.97e-21$ y $\sim 8.16e-18$, respectivamente, y su validación contra el corpus etiquetado.

El algoritmo de Viterbi, como técnica de programación dinámica, selecciona en cada paso la etiqueta que maximiza la probabilidad conjunta basada en las probabilidades de emisión y transición extraídas de un corpus de 20 oraciones y 137 tokens. La incorporación de suavizado de Laplace asegura robustez al asignar probabilidades no nulas a palabras y transiciones no vistas, mientras que el modelado de los estados <START> y <END> captura correctamente las transiciones iniciales y finales de las oraciones. Estas mejoras, junto con el filtrado de estados relevantes y la generación de métricas como cobertura (100% para emisiones, 7.91% para transiciones) y entropía (6.4595 bits para emisiones, 4.8188 bits para transiciones), hacen que el modelo sea adecuado para escenarios donde las probabilidades del corpus reflejan las dependencias lingüísticas, logrando una representación precisa de las estructuras morfosintácticas según el estándar EAGLES.

No obstante, el modelo presenta limitaciones inherentes al tamaño reducido del corpus y al uso de bigramas, que solo consideran la etiqueta anterior, lo que puede no ser suficiente para resolver ambigüedades léxicas en contextos más complejos. Para alcanzar mayor exactitud, se podrían emplear modelos avanzados como Redes Neuronales o Transformers, que capturan contextos más amplios, aunque requieren grandes cantidades de datos y recursos computacionales. Alternativamente, mejoras como la incorporación de trigramas, suavizado Kneser-Ney, o la combinación con reglas lingüísticas podrían optimizar el rendimiento sin la complejidad de enfoques neuronales.

En conclusión, el algoritmo de Viterbi implementado en este HMM de bigramas es idóneo para encontrar secuencias óptimas de etiquetas morfosintácticas en escenarios donde la velocidad y la eficiencia son prioritarias, como lo demuestra la generación de siete tablas Excel con probabilidades,

matrices de Viterbi, y rutas de etiquetas, junto con un análisis detallado de los resultados. Este trabajo establece una base sólida para el etiquetado morfosintáctico, con un enfoque claro y directo que permite futuras extensiones hacia modelos más robustos o corpus más extensos, manteniendo un equilibrio entre precisión y simplicidad computacional.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en la búsqueda de información:

- Tema 1. Introducción al procesamiento del lenguaje natural. Tema 2. El texto como dato. Tema 3. Etiquetado morfosintáctico (POS tagging). Tema 4. Análisis sintáctico. Tema 5. Modelado del lenguaje.
- Clases virtuales con el profesor Ing. Diego Osorio Reina.
- Material de apoyo del aula ubicado en Documentación.
- Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing* (3rd ed.). Prentice Hall.
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257-286.
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.
- Toutanova, K., Klein, D., Manning, C. D., & Singer, Y. (2003). Feature-rich part-of-speech tagging with a cyclic dependency network. *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology (NAACL-HLT)*, 173-180.
- Bosch, A., & Moreno, A. (2005). FreeLing: An open-source suite of language analyzers. *Proceedings of the 4th International Conference on Language Resources and Evaluation (LREC)*.
- EAGLES (1996). *Recommendations for the Morphosyntactic Annotation of Corpora*. Expert Advisory Group on Language Engineering Standards.